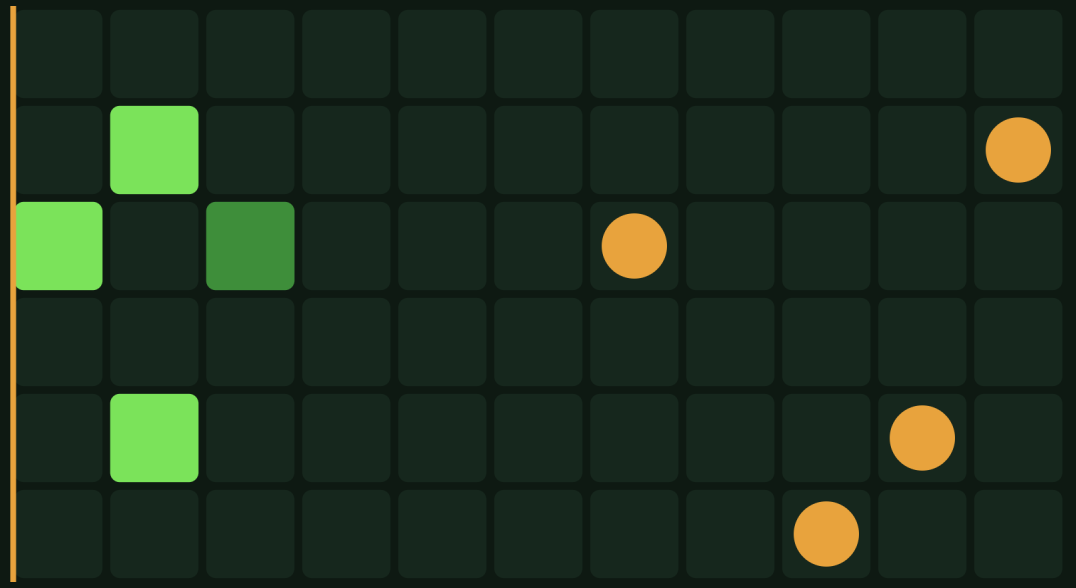
A SIX-HOUR AI CHALLENGE

AUTOPILOT

Write the agent that defends the base.

One Python file. Six hours. Two leaderboards.



[Event name] · [Date] · [Venue]

THE CHALLENGE

Your code defends the base

Enemies attack

Clowns, monks, wizards and pirates walk down six lanes toward your base.

Your agent decides

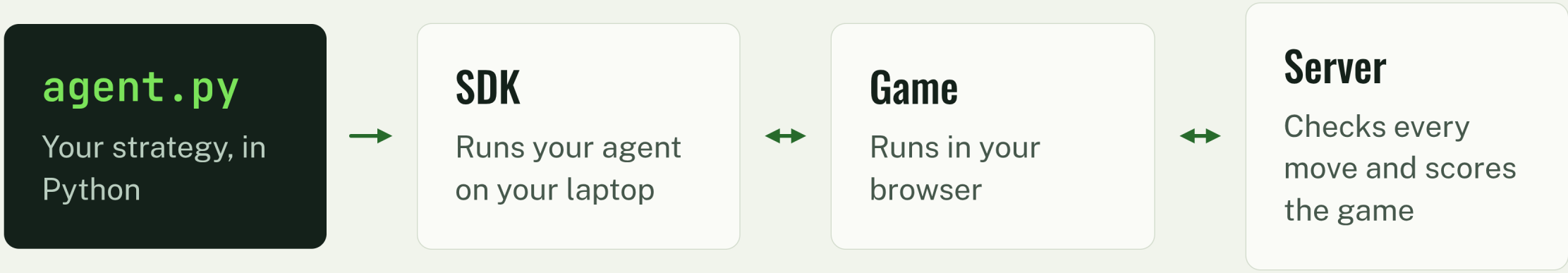
Your agent.py chooses which defences to place, and where, while the game is frozen.

The server scores

Every game is scored out of 100. Published runs go on the leaderboards.

HOW IT WORKS

How the pieces fit



At every pause the game asks the SDK, the SDK runs your **get_reinforcements(state)**, and the server's referee checks the defences you chose before the game places them.

THE DAY

Your six hours

[__:__] – [__:__]

Build

Practise with PLAY. Put your best agent on the live board with PUBLISH PUBLIC, as often as you like.

[__:__] – [__:__]

Final run

PUBLISH PRIVATE opens. Each team plays the hidden test cases once.

[__:__]

Results

The final leaderboard on the hidden test cases is revealed.

GETTING STARTED

Set up in five minutes

1 Get your login

The organisers send your team a 4-digit team ID and a password.

2 Install the SDK

macOS, Linux

```
curl -fsSL https://13-200-183-134.sslip.io/install.sh | sh
```

Windows

```
irm https://13-200-183-134.sslip.io/install.ps1 | iex
```

3 Start and log in

Press Enter to use the starter agent.py, then type your team ID and password. Run 4 VERIFY, then 1 PLAY.

4 Open the game

13-200-183-134.sslip.io in Chrome or Firefox. Type your team ID, press ▶, and click Allow when the browser asks to reach other apps.

GETTING STARTED

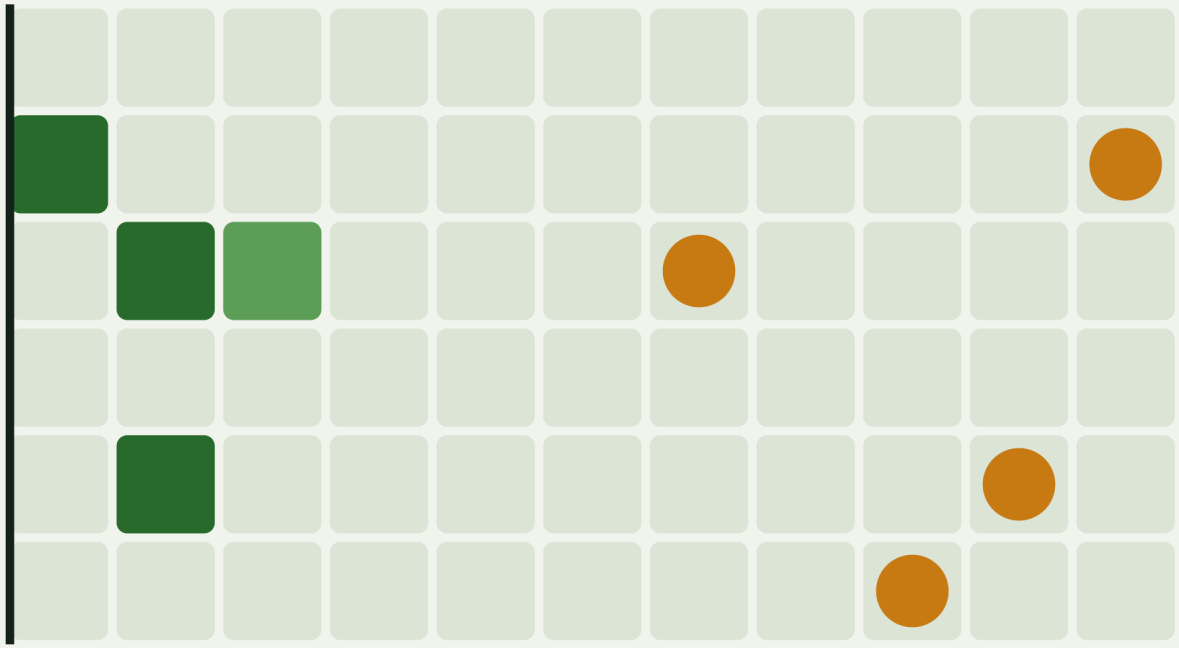
The SDK menu

Key	Option	What it does
1	PLAY	Practise one public test case. Never ranked.
2	PUBLISH PUBLIC	Every public test case once, onto the live board.
3	PUBLISH PRIVATE	Every hidden test case once, for the final board.
4	VERIFY	Checks agent.py and tries it on a sample board.
A · L · Q	Other	Another agent.py · switch team · quit

Ctrl+C returns to the menu. Edits to agent.py apply from the next pause.

THE GAME

The battlefield



Your base · x = 0

x = 10 · enemies enter

11 × 6 tiles

Columns x 0–10, rows y 0–5.

One lane per row

Enemies walk left from x = 10 and never change rows.

Lose

Any enemy crosses column 0.

Win

Every enemy in the wave defeated. No time limit.

THE GAME

Your four defences

`defenceId 0`

Penguin

250 coins
200 health

Waits until hit, then slides down its row, dealing 500 to every enemy it touches. One use.

`defenceId 1`

Soldier

200 coins
150 health

Shoots along its row at enemies on its tile or in front. About 40 damage a second.

`defenceId 2`

Archer

300 coins
100 health

Shoots into any row: lowest row first, then the enemy nearest your base. Fragile.

`defenceId 3`

Shield

150 coins
700 health

No attack. Enemies stop to hit it, so it protects the units behind it.

THE GAME

Four kinds of enemy

Enemy	Health	Crosses in	Attack
Clown · 0	200, regenerates	~7.5 s	Throws from ~3 tiles, 30 a throw
Monk · 1	280	~2.1 s	Melee, 40 a strike
Wizard · 2	500, regenerates	~5 s	Beam from ~5 tiles, 50 a second
Pirate · 3	30	~1.9 s	Explodes at the first defence: 200 to all within ~1.3 tiles

Clowns and wizards regain 15 health a second, so slow damage may never finish them. A pirate killed early still explodes where it falls.

WRITING YOUR AGENT

When your agent is called

`phase = "start"`

The opening

When the first enemy appears, before it moves. You get the whole wave and its spawn order.

→

`phase = "pause"`

Every 3 seconds

of game time after your last decision, until the run ends.

→

`timeBudgetSeconds = 7`

7 s to answer

The battle stays frozen. Too slow, or a crash: nothing is placed and the game goes on.

A level with reinforcements turned off has only the opening call.

WRITING YOUR AGENT

What your agent sees

state key	What it holds
phase, timeBudgetSeconds	"start" or "pause"; 7
counts	enemies on the field, still to come, wave size
defences	each one’s id, type, health, maxHealth, position
enemies	the same, plus isAttacking and isDead
budget	remaining coins, spent, unlocked, total
unitCosts	the cost of each defenceId

The opening also has **upcomingEnemies** (the whole wave), **spawnSettings** (spawn gaps and order) and **gameSettings**.

WRITING YOUR AGENT

Your agent is one function

```
def get_reinforcements(state):
    coins = state["budget"]["remaining"]
    cost = state["unitCosts"]["1"]  # a soldier
    taken = {(d["position"]["x"], d["position"]["y"])
             for d in state["defences"] if d["health"] > 0}
    cards = []
    for enemy in state["enemies"]:
        row = enemy["position"]["y"]
        if enemy["isDead"] or (1, row) in taken or coins < cost:
            continue
        cards.append({"defenceId": 1,
                      "defenceGridPosition": {"x": 1, "y": row}})
        taken.add((1, row))
        coins -= cost
    return cards
```

Return a list of cards

An empty list places nothing.

Each card is a defence and a tile

defenceId 0–3, x 0–10, y 0–5.

Start from the starter

Its LLM plans; a rule like this one is the fallback.

WRITING YOUR AGENT

Coins and the referee

Starting coins on the public test cases

+50

for every enemy you defeat

The referee drops a card when

- the tile is off the board
- the defenceId isn’t 0 to 3
- the tile already has a living defence
- the same tile is used twice in one answer
- you can’t afford it with the coins left

A dropped card never stops the others. The SDK window shows each refusal and why.

WRITING YOUR AGENT

The LLM, and the rules

The organisers' LLM

```
from autopilot_sdk.llm import LLMClient
```

- No key or URL to set: it comes with your login
- 60 calls a minute, shared by your whole team
- Mark a function with @tool and the LLM can call it

What agent.py may use

- The Python standard library and autopilot_sdk
- No network, except the LLM
- No starting other programs
- 4 VERIFY checks your file before you play

SCORES AND LEADERBOARDS

How a game is scored

25	25	25	10	15
Win	Kills	Defences standing	Speed	Coins left

Up to 100 for a won game. A lost game scores only its kills: up to 25.

A published run scores the sum of its games. A test case you don’t play counts 0.

SCORES AND LEADERBOARDS

Two leaderboards

LIVE · PUBLIC TEST CASES

PUBLISH PUBLIC

- Every public test case once
- Publish as often as you like
- Your best run counts
- Updates live at 13-200-183-134.sslip.io/leaderboard

FINAL · HIDDEN TEST CASES

PUBLISH PRIVATE

- Every hidden test case once
- Opens when the build phase ends
- One run per team
- Revealed by the organisers at the end

PLAY is practice: it never reaches a leaderboard.

THE FINAL RUN

One run. One agent.py.

- Your first PUBLISH PRIVATE locks your team to that agent.py.
- Stopped halfway? Restart with the same file and it carries on where it stopped.
- A different file is refused, and a finished run can’t be played again.
- Agree as a team which agent.py to publish, and publish from one laptop.

FAIR PLAY

Every run is checked

A frozen agent

A published run plays the copy of agent.py it started with. Edits during the run change nothing.

The same agent everywhere

The server checks that every hidden test case is played by the same agent.py.

Everything is recorded

Each run’s agent.py and run log go to the organisers. Keep your autopilot-runs folder.

Results are verified

[Results that don’t match what the server saw are flagged. The organisers may replay the top agents.]

FAIR PLAY AND TIPS

Tips for a strong start

01 Start with 4 VERIFY, then 1 PLAY on TC01.

02 Always return a list. An empty one is a valid answer.

03 Answer inside 7 seconds. Keep a fallback for a slow LLM.

04 Spend from budget remaining, not from a guess.

05 Watch the SDK window: it shows every refusal and why.

06 The hidden test cases differ. Don’t tune only to the public ones.

READY?

Good luck. Defend the line.

Where	Link
Game	https://autopilot.gdgnsut.com/
Downloads	https://autopilot.gdgnsut.com/download/
Leaderboard	https://autopilot.gdgnsut.com/leaderboard

DOWNLOADS

REGISTRATION